

Phase 3 - Spring Boot API Foundation

Phase objective

The objective of Phase 3 was to add a reliable Java backend foundation to the Quiz Learning Platform without changing the completed React quiz experience.

At the end of this phase, the repository includes:

- A Java 21 Spring Boot application under `apps/api`
- A Maven build managed through the Maven Wrapper
- Spring Web and request-validation foundations
- A working `GET /api/health` endpoint
- An immutable health-response model implemented as a Java record
- Externalized application name and server-port configuration
- A Spring application-context test
- A focused MVC controller test with `MockMvc`
- A Java 21 backend quality job in GitHub Actions
- Backend development commands in the project documentation
- Root README instructions for running and testing the API

This phase intentionally does not generate quizzes yet. Its purpose is to establish a tested backend boundary before adding quiz-generation contracts, services, provider interfaces, or AI integrations.

Architecture decisions used in this phase

Keep the monorepo

The project keeps the frontend and backend in one repository:

```
quiz-learning-platform/
|-- .github/
|   |-- workflows/
|   |-- ci.yaml
|-- apps/
|   |-- api/
|   |-- web/
|-- docs/
|-- package.json
|-- README.md
```

The web application continues to use npm workspaces. The API is managed independently with Maven.

This structure provides:

- One pull request for cross-application changes
- One CI workflow with isolated frontend and backend jobs
- Shared project documentation
- Independent build tools for each application
- A clear path for future API integration

Target Java 21

The project architecture targets Java 21 even though the development machine currently runs Java 26.

The target version represents the language and bytecode level the project promises to support. The local runtime may be newer, but CI validates the backend with Java 21.

```
Local machine runtime -> Java 26
Project target        -> Java 21
CI runtime            -> Java 21
```

This reduces the risk of accidentally using Java features that are unavailable in the selected project runtime.

Use Spring Boot as the backend framework

Spring Boot provides the foundation for:

- HTTP request handling
- JSON serialization
- Dependency injection
- Externalized configuration
- Validation
- Testing support
- Embedded application startup

The backend begins as a modular monolith. Microservices are not needed for the first product version.

Keep the health endpoint simple

The first endpoint is intentionally small:

GET /api/health

It proves that:

- The application starts
 - Spring discovers the controller
 - HTTP routing works
 - Java objects are serialized to JSON
 - Configuration can be injected
 - The endpoint contract can be tested
 - CI can compile and test the backend
-

Generated backend structure

```
apps/api/
|-- .mvn/
|-- src/
|   |-- main/
|   |   |-- java/com/quizlearningplatform/api/
|   |   |   |-- QuizLearningPlatformApiApplication.java
|   |   |   |-- health/
|   |   |       |-- HealthController.java
|   |   |       |-- HealthResponse.java
|   |   |-- resources/
|   |       |-- application.yaml
|   |-- test/
|       |-- java/com/quizlearningplatform/api/
|       |   |-- QuizLearningPlatformApiApplicationTests.java
|       |   |-- health/
|       |       |-- HealthControllerTest.java
|-- mvnw
|-- mvnw.cmd
`-- pom.xml
```

The package path and the package declaration must match.

For example:

File path:

src/test/java/com/quizlearningplatform/api/health/HealthControllerTest.java

Package declaration:

```
package com.quizlearningplatform.api.health;
```

A mismatch may produce editor errors or prevent the test from being discovered as expected.

Maven project fundamentals

What Maven does

Maven is the backend build and dependency-management tool.

It reads `pom.xml` to understand:

- Project identity
- Java configuration
- Dependencies
- Build plugins
- Test tooling
- Packaging rules

Common lifecycle commands used in this phase include:

```
mvnw.cmd test
mvnw.cmd clean test
mvnw.cmd spring-boot:run
```

Project coordinates

A Maven project uses coordinates such as:

```
groupId:    com.quizlearningplatform
artifactId: api
version:    0.0.1-SNAPSHOT
```

groupId The `groupId` identifies the organization or logical namespace that owns the artifact.

`com.quizlearningplatform`

artifactId The `artifactId` identifies this specific build artifact.

`api`

version The version identifies the current artifact revision.

`0.0.1-SNAPSHOT`

`SNAPSHOT` indicates active development rather than a final immutable release.

Why use the Maven Wrapper

The repository contains:

```
mvnw      -> Unix-like systems
mvnw.cmd  -> Windows
```

The wrapper gives the project a repeatable Maven entry point. Developers and CI do not need to rely only on whichever global Maven command happens to be installed.

From Windows, the backend commands are run with:

```
mvnw.cmd test
```

On Linux-based CI, the equivalent is:

```
./mvnw test
```

Maven runs from the project directory

Maven searches for `pom.xml` in the current working directory unless another project file is explicitly provided.

This failed from the repository root:

```
apps\api\mvnw.cmd --batch-mode test
```

The wrapper was found, but Maven still used the repository root as its working directory and could not find `pom.xml` there.

The recommended approach is:

```
cd apps\api
mvnw.cmd --batch-mode test
```

An alternative from the repository root is:

```
apps\api\mvnw.cmd --batch-mode -f apps\api\pom.xml test
```

The `-f` option points Maven to the correct project file.

Spring Boot application entry point

Spring Initializr generated the main application class:

```
package com.quizlearningplatform.api;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class QuizLearningPlatformApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            QuizLearningPlatformApiApplication.class,
            args
        );
    }
}
```

@SpringBootApplication

This annotation is the main Spring Boot application marker. It enables the application configuration and allows Spring to discover components below the application package.

Because the main class is in:

```
com.quizlearningplatform.api
```

Spring can discover the health package below it:

```
com.quizlearningplatform.api.health
```

This package organization matters. Feature packages should remain underneath the root application package unless component scanning is configured differently.

SpringApplication.run

This starts the Spring application context and the embedded web application.

Conceptually:

```
main method
  |
  v
SpringApplication.run
  |
  v
Create application context
  |
  v
Discover components
  |
  v
Start embedded server
```

Health response model

The endpoint returns an immutable Java record:

```
package com.quizlearningplatform.api.health;

public record HealthResponse(
    String status,
    String application
) {
}
```

Why a record fits this response

HealthResponse is a data carrier. It represents a response contract and does not require mutable state.

A record automatically provides behavior such as:

- A canonical constructor
- Accessor methods
- equals
- hashCode
- toString

The accessors are named after the components:

```
response.status();
response.application();
```

Record compared with a traditional class

A traditional version would require more code:

```
public final class HealthResponse {
    private final String status;
    private final String application;

    public HealthResponse(
        String status,
        String application
    ) {
        this.status = status;
        this.application = application;
    }

    public String getStatus() {
```

```

        return status;
    }

    public String getApplication() {
        return application;
    }
}

```

The record expresses the same data-focused intent more directly.

JSON serialization

Spring converts the record into JSON:

```

{
  "status": "UP",
  "application": "quiz-learning-platform-api"
}

```

The Java model is the server-side representation. JSON is the HTTP response representation.

Health controller

The controller exposes the first API endpoint:

```

package com.quizlearningplatform.api.health;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/health")
public class HealthController {

    private final String applicationName;

    public HealthController(
        @Value("${spring.application.name}")
        String applicationName
    ) {
        this.applicationName = applicationName;
    }

    @GetMapping
    public HealthResponse getHealth() {
        return new HealthResponse(
            "UP",
            applicationName
        );
    }
}

```

@RestController

@RestController marks the class as an HTTP controller whose returned values are written to the response body.

The method returns a Java object:

```
return new HealthResponse("UP", applicationName);
```

Spring serializes it instead of searching for an HTML view.

@RequestMapping

The class-level route is:

```
@RequestMapping("/api/health")
```

This becomes the base path for methods in the controller.

@GetMapping

The method-level annotation is:

```
@GetMapping
```

It maps HTTP GET requests. Because it does not add another path segment, the final route is:

GET /api/health

Controller responsibility

The controller should remain focused on the HTTP boundary.

In this endpoint, it:

- Receives a GET request
- Creates the response model
- Returns the response

Future quiz-generation business logic should not be implemented directly in a controller. It will belong in application services and provider abstractions.

Dependency injection and configuration injection

The controller requires the application name:

```
private final String applicationName;
```

Spring supplies it through the constructor:

```
public HealthController(  
    @Value("${spring.application.name}")  
    String applicationName  
) {  
    this.applicationName = applicationName;  
}
```

Why constructor injection

Constructor injection makes the dependency explicit.

The controller cannot be created without an application name:

```
new HealthController("quiz-learning-platform-api");
```

Benefits include:

- Required dependencies are visible
- Fields can remain `final`
- The object is valid immediately after construction
- Testing is easier
- Hidden mutation is avoided

Why not field injection

This style was intentionally avoided:

```
@Value("${spring.application.name}")
private String applicationName;
```

Field injection hides the dependency and requires the framework to mutate the object after construction.

@Value

The expression:

```
@Value("${spring.application.name}")
```

asks Spring to resolve the `spring.application.name` property from the application environment.

This phase uses `@Value` because the controller needs one simple property. A future feature with multiple related configuration values may use a dedicated configuration-properties class instead.

Externalized configuration

The API configuration is stored in:

`src/main/resources/application.yaml`

```
spring:
  application:
    name: quiz-learning-platform-api

server:
  port: ${SERVER_PORT:8080}
```

Application name

```
spring:
  application:
    name: quiz-learning-platform-api
```

This value is injected into the health controller and returned by the endpoint.

Moving it out of Java code prevents the controller from owning environment configuration.

Server port expression

```
server:
  port: ${SERVER_PORT:8080}
```

This means:

Use `SERVER_PORT` when it exists.
Otherwise use `8080`.

The syntax contains:

```
${PROPERTY_OR_VARIABLE:DEFAULT_VALUE}
```

Running on another port

On Windows Command Prompt:

```
set SERVER_PORT=9090
mvnw.cmd spring-boot:run
```

The API then starts at:

http://localhost:9090

The variable can be cleared afterward:

```
set SERVER_PORT=
```

Why externalized configuration matters

The same application build can run in different environments without source-code changes.

Local development -> port 8080

Alternative local run -> port 9090

Deployment platform -> platform-provided port

Configuration changes. Application logic remains the same.

Manual endpoint verification

Start the backend from apps/api:

```
mvnw.cmd spring-boot:run
```

With the application running, use a second terminal:

```
curl.exe http://localhost:8080/api/health
```

Expected compact response:

```
{"status":"UP","application":"quiz-learning-platform-api"}
```

Whitespace is not part of the JSON contract. The compact response is equivalent to a formatted response.

Stop the server with:

Ctrl + C

Manual verification proves that a real server can start and receive an HTTP request. Automated tests provide repeatable verification during development and CI.

Application-context test

Spring Initializr generated an application test similar to:

```
package com.quizlearningplatform.api;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class QuizLearningPlatformApiApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Purpose of @SpringBootTest

This test loads the Spring application context.

It catches problems such as:

- Invalid configuration

- Missing required beans
- Component-creation failures
- Dependency-injection failures
- Application startup incompatibilities

The empty test method is intentional. The assertion is that the application context can start successfully.

Scope

This is a broad integration-style framework test. It loads more application infrastructure than a focused controller test.

@SpringBootTest -> broad application context

@WebMvcTest -> focused web layer

Both tests provide different value.

Focused MVC controller test

The health controller has a focused web-layer test:

```
package com.quizlearningplatform.api.health;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(
    controllers = HealthController.class,
    properties = {
        "spring.application.name=quiz-learning-platform-api"
    }
)
class HealthControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void shouldReturnApiHealthStatus() throws Exception {
        mockMvc.perform(get("/api/health"))
            .andExpect(status().isOk())
            .andExpect(
                content().contentTypeCompatibleWith(
                    MediaType.APPLICATION_JSON
                )
            )
            .andExpect(jsonPath("$.status").value("UP"))
            .andExpect(
                jsonPath("$.application")
                    .value("quiz-learning-platform-api")
            )
    }
}
```

```

    );
}
}

```

@WebMvcTest

@WebMvcTest creates a focused Spring MVC test context for the selected controller.

```
@WebMvcTest(controllers = HealthController.class)
```

This is faster and narrower than loading the entire application for every controller contract test.

Test-specific property

The annotation provides:

```
properties = {
    "spring.application.name=quiz-learning-platform-api"
}
```

This makes the controller's configuration dependency explicit inside the test.

The test does not silently depend on whichever value happens to exist in the normal application configuration.

MockMvc

MockMvc executes the Spring MVC request-handling pipeline without opening a real network port.

```
mockMvc.perform(get("/api/health"))
```

It lets the test verify routing, response status, content type, serialization, and JSON content.

Contract assertions

The test verifies:

```

HTTP status    -> 200 OK
Content type   -> JSON-compatible
status         -> UP
application    -> quiz-learning-platform-api

```

Checking only status 200 would be insufficient. An endpoint could return the wrong JSON body while still returning a successful status.

JSONPath

The expression:

```
jsonPath("$.status")
```

selects the root-level status property.

The expression:

```
jsonPath("$.application")
```

selects the root-level application property.

Test source-set organization

Production and test code belong in different Maven source sets.

```

src/main/java -> production Java code
src/test/java -> test Java code

```

Test libraries are generally available only to the test source set.

The controller test initially had an incorrect physical location. Its package declared:

```
package com.quizlearningplatform.api.health;
```

but the file was placed directly under:

```
src/test/java/com/quizlearningplatform/api/
```

The corrected location is:

```
src/test/java/com/quizlearningplatform/api/health/HealthControllerTest.java
```

Test discovery result

Before correcting the location, Maven reported only one test:

```
Tests run: 1
```

After correcting and saving the file, the complete suite should include:

```
QuizLearningPlatformApiApplicationTests  
HealthControllerTest
```

and report:

```
Tests run: 2  
Failures: 0  
Errors: 0  
BUILD SUCCESS
```

A successful build is not always proof that every intended test ran. Test counts and test-class names should also be reviewed.

Clean builds and test commands

Run tests

```
mvnw.cmd test
```

This compiles necessary sources and runs tests.

Clean and test

```
mvnw.cmd clean test
```

The `clean` phase removes previous build output before compilation and testing.

This is useful after moving files or when stale compiled output may hide a problem.

Batch mode

```
mvnw.cmd --batch-mode clean test
```

Batch mode is appropriate for non-interactive environments such as CI.

Build output directory

Maven generates compiled output under:

```
target/
```

This directory is build output and should not be treated as source code.

Understanding the Java-agent warnings

Tests executed successfully on the local Java 26 runtime but displayed Mockito and Byte Buddy warnings about dynamic agent attachment.

The important build result was:

```
Failures: 0
Errors: 0
BUILD SUCCESS
```

The warnings did not mean the implementation failed. They indicated a future compatibility concern in the testing toolchain when running on a newer JDK.

For this phase:

- No production failure occurred
- The tests completed successfully
- CI validates the project with Java 21
- The warning can be revisited when Mockito configuration becomes an active need

A professional debugging habit is to distinguish:

```
Warning -> operation completed, but attention may be needed
Error   -> operation failed
```

Backend continuous integration

The existing workflow originally contained only the frontend job.

Phase 3 added a separate backend job:

```
api-quality:
  name: API Quality Checks
  runs-on: ubuntu-latest

  steps:
    - name: Check out repository
      uses: actions/checkout@v4

    - name: Set up Java
      uses: actions/setup-java@v6
      with:
        distribution: temurin
        java-version: "21"
        cache: maven
        cache-dependency-path: apps/api/pom.xml

    - name: Make Maven Wrapper executable
      run: chmod +x apps/api/mvnw

    - name: Run API tests
      working-directory: apps/api
      run: ./mvnw --batch-mode test
```

Independent quality jobs

The workflow now has two jobs:

```
web-quality -> React and TypeScript quality checks
api-quality -> Java and Spring Boot tests
```

The jobs are independent, so a backend failure does not hide whether the frontend passed, and vice versa.

Java version in CI

`java-version: "21"`

This verifies the architecture target instead of relying on the newer Java version installed locally.

Maven dependency cache

`cache: maven`

`cache-dependency-path: apps/api/pom.xml`

The cache configuration uses the backend project file to calculate dependency-cache changes in the monorepo.

Linux wrapper permission

The repository includes `mvnw` for Linux-based CI. The workflow ensures it is executable:

`run: chmod +x apps/api/mvnw`

Working directory

`working-directory: apps/api`

This ensures Maven runs in the directory containing `pom.xml`.

It prevents the same missing-project error encountered when Maven was run from the repository root without `-f`.

Documentation added in this phase

Root README

The README now explains:

- Where the backend is located
- How to run the API
- The health endpoint contract
- How to run backend tests
- How to validate both applications

Development commands

The new file:

`docs/development-commands.md`

centralizes commands for:

- Installing frontend dependencies
- Running the React application
- Formatting, linting, type checking, testing, and building the web app
- Running the Spring Boot API
- Testing the health endpoint
- Running Maven tests
- Using a different API port
- Performing full local validation

Centralized commands reduce setup ambiguity for future contributors and for the developer returning to the project later.

Problems encountered and lessons learned

Problem: Maven Wrapper was not recognized

The command was executed from the repository root:

```
mvnw.cmd test
```

Windows could not find the wrapper because it exists under `apps/api`.

Lesson A command name without a path is resolved from the current directory and the system path.

Correct options:

```
cd apps\api
mvnw.cmd test
```

or:

```
apps\api\mvnw.cmd -f apps\api\pom.xml test
```

Problem: Maven could not find a project

The wrapper was called with a relative path, but Maven still executed from the repository root:

There is no POM in this directory

Lesson Finding the wrapper script and finding `pom.xml` are separate concerns. Maven uses the current working directory unless `-f` is provided.

Problem: Test code was placed in the wrong source tree

A test file under `src/main/java` cannot rely on test-scoped dependencies in the expected way.

Lesson Use:

```
src/main/java -> production
src/test/java -> tests
```

Problem: Package declaration and directory did not match

The test declared the `health` package but was physically one directory too high.

Lesson Keep Java package declarations aligned with directory paths. This improves compiler behavior, IDE navigation, and test discovery.

Problem: Build passed but only one test ran

The first Maven result showed:

```
BUILD SUCCESS
Tests run: 1
```

The intended controller test had not been included.

Lesson Do not stop at `BUILD SUCCESS`. Review:

- Test count
- Test-class names
- Failure count
- Error count

Problem: Line-ending-only frontend changes appeared

Unrelated React files appeared modified while working on the backend branch.

Lesson Use targeted inspection before staging:

```
git diff --ignore-space-at-eol -- apps/web
```

Restore unrelated paths rather than including them in the backend commit.

Problem: Local Java and project Java were different

The computer used Java 26 while the project target was Java 21.

Lesson Separate these ideas:

Installed runtime

Project compilation target

CI runtime

The CI job is the authoritative guard for the selected project runtime.

Backend request lifecycle learned in this phase

For the health endpoint:

```
Client sends GET /api/health
      |
      v
Spring MVC matches HealthController
      |
      v
getHealth() creates HealthResponse
      |
      v
Spring serializes record to JSON
      |
      v
Client receives HTTP 200 + JSON
```

For the focused test:

```
Test creates MockMvc request
      |
      v
Spring MVC routes request
      |
      v
Controller returns HealthResponse
      |
      v
Serializer creates JSON
      |
      v
Assertions verify public contract
```

This is the foundation for the future quiz-generation endpoint.

Separation of concerns

Phase 3 establishes several boundaries:

Configuration

`application.yaml`

Owns environment-adjustable values.

HTTP layer

`HealthController`

Owns route handling and response delivery.

Response contract

`HealthResponse`

Owns the shape of returned health data.

Framework startup

`QuizLearningPlatformApiApplication`

Owns application bootstrapping.

Verification

`QuizLearningPlatformApiApplicationTests`

`HealthControllerTest`

Own startup and HTTP-contract verification.

The next phase will add more boundaries:

Controller -> Application service -> QuizGenerator interface -> Provider

Testing strategy established

The backend testing pyramid begins with two complementary tests.

Context test

Purpose:

Can the Spring application start?

Tool:

`@SpringBootTest`

MVC slice test

Purpose:

Does the endpoint satisfy its HTTP and JSON contract?

Tools:

`@WebMvcTest`

`MockMvc`

Future tests

The next phases should add:

- Request-validation tests
- Pure service unit tests
- Fake provider tests
- Controller tests for success and error responses
- JSON contract tests
- Provider-mapping tests without real AI calls

CI must remain deterministic and must not require an OpenAI API key.

Git workflow used for the phase

The work was developed on:

gabs/03-spring-boot-api-foundation

A checkpoint commit was pushed before the final documentation was added:

```
git commit -m "feat: add Spring Boot API foundation"
git push -u origin gabs/03-spring-boot-api-foundation
```

The complete phase should include only related paths such as:

```
.github/workflows/ci.yaml
apps/api/
README.md
docs/development-commands.md
docs/phases/03-spring-boot-api-foundation.md
docs/phases/03-spring-boot-api-foundation.pdf
```

Unrelated frontend source changes should not be included.

Development command reference

Start the frontend

From the repository root:

```
npm run dev:web
```

Validate the frontend

```
npm run quality:web
```

Start the API

```
cd apps\api
mvnw.cmd spring-boot:run
```

Test the API

```
cd apps\api
mvnw.cmd clean test
```

Test the endpoint

With the API running:

```
curl.exe http://localhost:8080/api/health
```

Full local validation

```
npm run quality:web
cd apps\api
mvnw.cmd --batch-mode clean test
cd ..\..
```

Inspect pending changes

```
git status
git diff --stat
```

Interview questions and model answers

Why did you use Spring Boot for this backend?

Spring Boot provides a structured foundation for REST endpoints, dependency injection, configuration, validation, testing, and future AI-provider integration. It also fits the project's goal of developing enterprise Java skills while keeping the first backend deployment simple.

Why is the project targeting Java 21 while your machine uses Java 26?

The project target defines the supported runtime and language level. A newer local JDK can run the build, but CI uses Java 21 to prevent accidental use of newer language or platform features.

What problem does the Maven Wrapper solve?

It gives developers and CI a repository-controlled Maven entry point. The build does not depend only on a globally installed Maven command or version.

What is the difference between `groupId` and `artifactId`?

The `groupId` identifies the organization or namespace. The `artifactId` identifies the specific project artifact within that namespace.

What does `@SpringBootApplication` do?

It marks the main application configuration, enables Spring Boot application setup, and supports component discovery below the root package.

Why did you use a Java record for `HealthResponse`?

The response is an immutable data carrier with no mutable behavior. A record communicates that intent and automatically provides its constructor, accessors, equality, hashing, and string representation.

What is the difference between `@RestController` and `@GetMapping`?

`@RestController` marks a class as an HTTP controller whose return values are written to the response body. `@GetMapping` maps a specific method to HTTP GET requests.

Why use constructor injection?

Constructor injection makes required dependencies explicit, supports immutable fields, produces valid objects at construction time, and improves testability.

What does `${SERVER_PORT:8080}` mean?

It uses the `SERVER_PORT` value when available and falls back to `8080` otherwise.

Why use externalized configuration?

It allows the same compiled application to run in multiple environments with different settings without changing source code.

What is the difference between `@SpringBootTest` and `@WebMvcTest`?

`@SpringBootTest` loads the broad application context and verifies startup integration. `@WebMvcTest` loads a focused web-layer context for controller behavior.

Does `MockMvc` start a real server on port 8080?

No. It exercises Spring MVC request handling inside the test process without opening a real network port.

Why verify JSON properties instead of only checking HTTP 200?

A successful status does not guarantee the response body follows the required contract. JSON assertions verify both behavior and returned data.

Why must package declarations match directory paths?

Matching them keeps Java source organization consistent and prevents problems with compilation, IDE resolution, and test discovery.

Why can a successful build still require investigation?

A build may pass while an expected test was not discovered. Test counts, test-class names, failures, and errors must all be reviewed.

Why are frontend and backend CI checks separate jobs?

Separate jobs isolate failures, allow parallel execution, and make it immediately clear which application failed validation.

Why is working-directory: apps/api important in CI?

Maven needs to run in the directory containing `pom.xml`. The setting prevents a missing-project failure from the monorepo root.

Why did you not add a database in this phase?

The current backend is stateless and only establishes the API foundation. Browser `localStorage` already satisfies Version 1 score persistence. A database should be introduced only when product requirements justify it.

Explain-back answers

Why does a fallback Java target matter?

A stable project target makes the runtime promise explicit and allows CI to catch accidental use of unsupported APIs or language features.

Why does the health endpoint return the application name?

It confirms that configuration injection works and helps identify which application responded.

Why does the controller depend on configuration instead of hardcoding the name?

The controller should handle HTTP behavior, while environment-adjustable values should come from configuration.

Why is the controller test a contract test?

It verifies externally observable behavior: route, status, content type, and JSON fields. It does not depend on the method's private implementation details.

Why is `clean test` useful after moving a Java file?

It removes stale compiled output and forces Maven to rebuild the source sets from the corrected directory structure.

Why did CI use Java 21 instead of the local Java 26 runtime?

CI verifies the selected architecture target and provides a consistent environment for all contributors.

Practice exercises

Complete these before Phase 4 or during review.

Exercise 1 - Add a version property

Add an application version to configuration:

```
application:
  version: 0.1.0
```

Return it from the health endpoint:

```
{
  "status": "UP",
  "application": "quiz-learning-platform-api",
  "version": "0.1.0"
}
```

Update the MVC test first, then update the implementation.

Exercise 2 - Write a direct unit test

Instantiate `HealthController` without Spring:

```
HealthController controller = new HealthController(
    "quiz-learning-platform-api"
);
```

Call `getHealth()` and verify the returned record values.

Compare the value of this unit test with the `MockMvc` contract test.

Exercise 3 - Add a second test property

Change the MVC test property to:

```
spring.application.name=test-api
```

Update the expected JSON value and explain why the test still passes independently from `application.yaml`.

Exercise 4 - Verify an unsupported method

Use `MockMvc` to send a POST request to `/api/health` and verify the response is not successful.

Exercise 5 - Add a query parameter

Create an optional `details` query parameter:

```
GET /api/health?details=true
```

Keep the original contract unchanged when the parameter is absent. Write tests before implementing the behavior.

Exercise 6 - Run from the repository root

Run Maven successfully from the root using -f:

```
apps\api\mvnw.cmd -f apps\api\pom.xml clean test
```

Explain why the wrapper path alone does not change Maven's working directory.

Exercise 7 - Break package alignment intentionally

Temporarily move a test file one directory higher without changing its package. Observe the IDE and Maven behavior, then restore it.

Exercise 8 - Inspect the dependency tree

Run:

```
mvnw.cmd dependency:tree
```

Identify the dependencies that provide Spring MVC, JSON serialization, and testing support.

Exercise 9 - Compare test scopes

Run only the context test, then only the controller test. Compare their logs and startup time.

Exercise 10 - Add a CI packaging check

After tests, add a Maven packaging step or replace the test command with an appropriate lifecycle command that also verifies packaging. Explain the additional assurance it provides.

Phase 3 final result

At the end of Phase 3, the product has two independently testable applications:

```
React frontend
|
| Future HTTP integration
v
Spring Boot API
```

The API foundation now provides:

- A stable Java package structure
- A repeatable Maven build
- A tested Spring application context
- A tested REST endpoint
- JSON serialization
- Configuration injection
- Environment-based port configuration
- Java 21 CI validation
- Development documentation

The project is ready to move from infrastructure to the first real backend feature.

Preparation for Phase 4

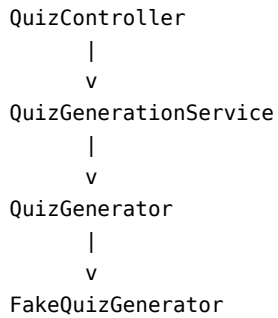
The next phase should implement the quiz-generation API contract without connecting to OpenAI yet.

Recommended sequence:

1. Define request and response DTOs.

2. Define difficulty and question-type enums.
3. Add Jakarta Validation rules.
4. Create the `QuizGenerator` interface.
5. Implement a deterministic `FakeQuizGenerator`.
6. Add a `QuizGenerationService`.
7. Add `POST /api/quizzes/generate`.
8. Test valid requests.
9. Test invalid requests and error responses.
10. Connect the React configuration form to the backend.

The target architecture becomes:



OpenAI should be introduced only after this contract and service flow are stable and fully tested.